

InfiniteVaultStrategy v2 & v2.1

Security review, change log and upgrade notes · Velodrome auto-compounder vaults (Optimism)

Date	2026-09-30
Scope	contracts/src/velodrome/InfiniteVaultStrategy.sol + interfaces; keeper velodrome-auto-compound.js; upgrade path via InfiniteVaultFactory
v1 (live before upgrade)	0x65c1D7Ae1F840f288c2CCD9a5871fca6497a65f2 (solc 0.8.28, runs 200, cancun)
v2	0xaE62D9A7682C1c8ca780dCe94E53D0153bb679dd · live on all vaults since 2026-09-30 · deploy tx 0x5fe7a911cd2028d0aa15aac425b3d19ad7a9ced7f52930b5a953f64121b2bad4 solc 0.8.26, via-IR, optimizer runs 200, evm cancun (FOUNDRY_PROFILE=velodrome) · runtime 24,368 bytes source sha256 7fd272b860f7c3967f461a8c38bbd1b8a0d6d988618edf5f8215de0a36f8719
v2.1 (this addendum)	0x36471C8330d4F3527997CC9B379cbF862eD42944 · deployed, pending DAO activation · deploy tx 0xc0369847a94bae264673ac9130e123e033ff863913a89d9175c61b62464582af same compiler settings · runtime 24,261 bytes · source sha256 f6fc8b6a8bae95b85745032e9e9254e1f0e04308c1ba91f18d63b6fc3bde482f
Governance	DAO Safe 0xb5dB6e5a301E595B76F40319896a8dbDc277CEfB (3-of-8): sole factory admin and owner of all vaults
Beacon / Factory	Beacon 0x7Ef3e66BD423B774E132fB15b19bd4c5216926e6 owned by factory proxy 0x88E883a28C82f7524C7aAd0a44309E190337D5b
Vaults	ITP/VELO 0x569D...2247, ITP/DHT 0xFCEa...CA6, ITP/wstETH 0x2811...563f, ITP/OP 0x8A2e...4b69, ITP/WBTC 0x3092...Cc11, ITP/USDC 0xc462...F276

Disclaimer. This is an internal review performed by the Infinite Trading team with AI assistance, backed by mainnet-fork tests. It is not an independent third-party audit and does not guarantee the absence of vulnerabilities.

1. Executive summary

Only the ITP/VELO vault was compounding. In the other five vaults every harvest swapped rewards but silently failed to add liquidity (H-01), so tokens piled up in the contracts. v2 fixes the root cause, deploys the backlog gradually with protections for ITP's thin liquidity, makes harvest-on-deposit/withdraw the default for all existing vaults without per-vault transactions, and guarantees deposits/withdrawals can never be blocked by a failing harvest. Storage layout is unchanged and was verified slot-by-slot on a mainnet fork against all six live vaults.

v2.1 addresses a vault whose Velodrome gauge was killed (ITP/DHT), which earned nothing at all (M-04): the vault now unstakes, collects the pool's swap fees directly and compounds them, and re-stakes automatically if the gauge is revived. Since the review, governance was also consolidated: the DAO Safe is the only account that can upgrade the vaults (I-01 resolved).

Severity	Count
High	2 (fixed)
Medium	4 (3 fixed, 1 mitigated)
Low	6 (4 fixed, 1 mitigated, 1 addressed by design)
Informational	6 (1 resolved)

2. Findings overview

ID	Severity	Title	Status
H-01	High	Harvested rewards were never compounded (5 of 6 vaults)	Fixed
H-02	High	Harvest-on-deposit priced new shares before the harvest	Fixed
M-01	Medium	A killed gauge bricks harvest (ITP/DHT)	Fixed
M-02	Medium	Permissionless harvest with same-block price quotes (manipulation / sandwich)	Mitigated
M-03	Medium	First-depositor share inflation	Fixed
M-04	Medium	Killed gauge: the vault earned zero yield	Fixed in v2.1
L-01	Low	Swallowed failures without observability	Fixed
L-02	Low	ITP/USDC reward route is dead	Mitigated

L-03	Low	Selling ITP into thin pools	Addressed (design)
L-04	Low	Harvest functions were not reentrancy-protected	Fixed
L-05	Low	Unreachable UUPS upgrade path	Fixed
L-06	Low	Incorrect stable-pool ratio math (dead code)	Fixed
I-01	Info	Centralisation: factory admins could upgrade every vault	Resolved
I-02	Info	Owner powers	Acknowledged
I-03	Info	Ineffective swap deadlines	Acknowledged
I-04	Info	Call fee paid to the batcher	Resolved in keeper
I-05	Info	Liveness trade-off of the TWAP guard	By design
I-06	Info	Route display views removed	Changed in v2.1

3. Detailed findings

High H-01 · Harvested rewards were never compounded (5 of 6 vaults)

Status: Fixed

`_addLiquidity()` passed `amountAMin/amountBMin = 99.5%` of the vault's *raw* token balances to `router.addLiquidity`. The router only consumes the pool-ratio-matched amount of one side, so whenever the two balances deviated $>0.5\%$ from the pool ratio the call reverted (`InsufficientAmount`). The revert was swallowed by an empty `catch {}`, `harvest()` succeeded, and the swapped tokens stayed idle. Because the next harvest used the *total* idle balances, the imbalance only grew and the vault could never recover.

ITP/VELO worked by coincidence (reward token is a pair token, swap happens in the same pool). All other vaults route $\text{VELO} \rightarrow \text{ITP} \rightarrow \text{X}$, which lands $>0.5\%$ off-ratio. On-chain before the fix: ITP/wstETH 11,195 ITP idle (5.4% off), ITP/USDC 13,654 ITP idle, ITP/WBTC 5,490 ITP idle (27.9% off), ITP/OP 21.7 ITP idle (1.1% off).

Fix: amounts are quoted with `router.quoteAddLiquidity` and the slippage minimums are derived from the quote. One-sided leftovers are converted through the pair with the closed-form optimal zap, capped for thin ITP liquidity (see L-03).

High H-02 · Harvest-on-deposit priced new shares before the harvest

Status: Fixed

`deposit()` read `_pool = balance()` *before* calling `_harvest()`, so harvested yield increased `balance()` after the share price was fixed. A depositor could capture pending rewards that belonged to existing holders (flash deposit $\rightarrow$ withdraw). Latent in v1 because `harvestOnDeposit` was false everywhere; it becomes exploitable once enabled, which v2 does by default.

Fix: harvest runs first, then the pool is measured. The same ordering is used for the new harvest-before-withdraw.

Medium M-01 · A killed gauge bricks harvest (ITP/DHT)

Status: Fixed

Velodrome killed the ITP/DHT gauge; `gauge.getReward()` and `gauge.deposit()` revert with `NotALive()`. In v1 every harvest of that vault reverted. With harvest-before-withdraw enabled this would have **blocked withdrawals**.

Fix (v2): inside the harvest path `getReward` and the re-stake are wrapped in `try/catch` (`GetRewardFailed`, `EarnFailed` events); idle want is still counted by `balance()`. `unpause()` uses the tolerant re-stake too. Verified on fork: withdraw succeeds on all 6 vaults including ITP/DHT. v2.1 goes further and makes such a vault productive again (see M-04).

Medium M-02 · Permissionless harvest with same-block price quotes (manipulation / sandwich)

Status: Mitigated

All `minOut` values are computed from `getAmountsOut` in the same transaction, which offers no protection against an attacker that moves the pool, calls `harvest()`, and unwinds atomically. v2 swaps larger amounts (idle leftovers), so the exposure increased.

Mitigation: before any swap or liquidity add, the pair's spot price is compared with Velodrome's on-chain TWAP (`pool.quote`, fee-adjusted). If it deviates by more than `MAX_TWAP_DEVIATION = 3%` the vault defers (`LiquidityDeferred`) and retries on the next harvest. Residual risk: reward swaps also traverse the ITP/VELO pool, which is only guarded for the ITP/VELO vault; the reward amounts involved are small (< 1 VELO/day per vault).

Medium M-03 · First-depositor share inflation

Status: **Fixed**

With `totalSupply() == 0` v1 minted `shares = amount`. An attacker can mint 1 wei of shares, donate `want` and make the next depositor's shares round to zero. It also applies to a vault that was fully withdrawn while `withdrawal-fee` residue remained in `balance()`.

Fix: the first deposit mints `MIN_SHARES = 1000` to `0x...dEaD` and `deposit` reverts on zero shares. Existing vaults (non-zero supply) are unaffected.

Medium M-04 · Killed gauge: the vault earned zero yield

Status: **Fixed in v2.1**

After Velodrome killed the ITP/DHT gauge, the vault kept all of its LP staked there: 64,624 LP, i.e. **99.8% of the pool's LP supply**. A killed gauge pays no emissions (`earned() = 0`), and because staked LP belongs to the gauge, the pool's 0.3% swap fees accrue to the gauge, which never forwards them once dead. On-chain the gauge had already accumulated 2.64 ITP + 0.34 DHT of fees that the vault can never claim. Depositors were therefore earning **nothing**, while new deposits reverted.

Fix (v2.1): a new view `gaugeAlive()` reads `gauge.voter().isActive(gauge)` (falls back to *alive* if unreadable, preserving v2 behaviour). When the gauge is dead, every harvest:

- withdraws all LP from the gauge (allowed by Velodrome for killed gauges),
- calls `pool.claimFees()` to collect the swap fees the now-unstaked LP has earned,
- compounds those `token0/token1` fees into LP through the existing quote-based add / capped zap, and keeps the LP unstaked.

Deposits into a dead-gauge vault are accepted and stay unstaked (earning swap fees). If Velodrome revives the gauge, the next harvest re-stakes everything automatically. All external calls in this path are `try/catch`, so it can never block deposits or withdrawals. Swap-fee income is not charged the performance fee (the fee logic only applies to the reward token), so 100% of it accrues to depositors.

Verified on fork: the first harvest moves all LP out of the gauge with no TVL loss; after simulated trading the next harvest compounds the fees and the vault's LP balance increases; a revived gauge is re-staked in full.

Low L-01 · Swallowed failures without observability

Status: **Fixed**

Every swap / liquidity failure was silently ignored, which is why H-01 went unnoticed. v2 emits `SwapFailed`, `AddLiquidityFailed`, `LiquidityDeferred`, `GetRewardFailed` and `EarnFailed`.

Low L-02 · ITP/USDC reward route is dead

Status: **Mitigated**

The vault's `outputToLp1Route` ends with ITP→USDC `stable=true`; that pool does not exist in the Velodrome factory, so rewards could never buy USDC. Routes are immutable (no setter).

Mitigation: the value-aware reward split sends rewards to the side whose route works; the capped zap then converts part of the ITP to USDC through the ITP/USDC volatile pair. Recommendation: migrate this vault if it should keep receiving deposits.

Low L-03 · Selling ITP into thin pools

Status: **Addressed (design)**

ITP liquidity is shallow (e.g. ITP/USDC ~412k ITP vs ~2.1k USDC). Converting idle ITP naively could move the ITP price materially. v2 controls this with three mechanisms:

- **Value-aware reward split** (`_rewardSplit0`): rewards are routed to the under-weight side first, so idle excess is consumed by rewards before any ITP is sold.
- **Zap cap:** a single zap sells at most `MAX_ZAP_BPS = 0.5%` of the pair's input-side reserve.
- **Zap cooldown:** zaps only run if the previous harvest is $\geq$ `ZAP_COOLDOWN = 1 hour` old, so repeated deposits/withdrawals/harvests cannot chain zaps; the 3% TWAP guard is a second backstop.

Fork test `test_ZapIsCappedForIlliquidPools` asserts the ITP price moves $\leq 1.5\%$ per harvest on every vault. Large backlogs (ITP/USDC, ITP/WBTC) are therefore deployed over several daily harvests.

Low L-04 · Harvest functions were not reentrancy-protected

Status: **Fixed**

`harvest()` / `harvest(address)` now carry `nonReentrant` (same ERC-7201 guard slot as v1's `ReentrancyGuardUpgradeable`).

Low L-05 · Unreachable UUPS upgrade path

Status: **Fixed**

v1 inherited `UUPSUpgradeable`, but every vault is a `BeaconProxy` whose ERC-1967 implementation slot is empty, so `upgradeToAndCall` always reverted. It was removed (it holds no storage), shrinking bytecode and attack surface. Upgrades happen exclusively through the factory's beacon.

Low L-06 · Incorrect stable-pool ratio math (dead code)

Status: **Fixed**

The stable-pair branch of v1's `_addLiquidity` mixed decimal normalisation and could produce wrong splits. No stable vault is deployed; the branch is replaced by the quote-based logic that is correct for both pool types.

Info I-01 · Centralisation: factory admins could upgrade every vault

Status: **Resolved**

At review time `InfiniteVaultFactory.DEFAULT_ADMIN_ROLE` was held by two **EOAs** (`0xe2c7Dc63...bD12`, `0xE4c0944C...a37C`). Either could call `upgradeBeacon` and replace the logic of all vaults, i.e. take custody of all deposits.

Resolution (2026-09-30, on-chain): `0xE4c0...a37C` revoked every role of `0xe2c7...bD12` (`DEFAULT_ADMIN`, `EXECUTOR`, `SETTINGS_ADMIN`, `VAULT_CREATOR`), granted `DEFAULT_ADMIN_ROLE` to the DAO Safe `0xb5dB6e5a...CEfB` (3-of-8), and then renounced its own role. The DAO Safe is now the **only** account able to upgrade vaults or the factory, and it also owns every vault. The operational roles (`EXECUTOR`, `SETTINGS_ADMIN`, `VAULT_CREATOR`) are unassigned and can be granted by the Safe when needed. Further hardening: a timelock in front of the Safe.

Info I-02 · Owner powers

Status: **Acknowledged**

The owner (DAO Safe) can set the performance fee up to 50% (`MAX_FEE`), withdrawal fee up to 5%, slippage up to 10%, pause/panic, and `inCaseTokensGetStuck` any token except `want` and vault shares (this includes pending reward and idle pair tokens). The owner is exempt from the withdrawal fee.

Info I-03 · Ineffective swap deadlines

Status: **Acknowledged**

`block.timestamp + swapDeadline` is always satisfied. Harmless on Optimism (sequencer ordering); kept for compatibility.

Info I-04 · Call fee paid to the batcher

Status: **Resolved in keeper**

Through `Multicall3` `msg.sender` is `Multicall3`, so `harvest()` would pay the call fee to a contract from which anyone can pull it. Batchers must use `harvest(address callFeeRecipient)`. The server keeper (`velodrome-auto-compound.js`) now discovers vaults from the factory and harvests all of them in one `Multicall3` transaction with `harvest(hotWallet)`; the app's *Harvest All* button does the same with `harvest(connectedWallet)`, so the caller receives the fee.

Info I-05 · Liveness trade-off of the TWAP guard

Status: **By design**

In fast markets (>3% spot vs TWAP) compounding is deferred; tokens remain in the vault and are retried next harvest. No funds are at risk.

Info I-06 · Route display views removed

Status: **Changed in v2.1**

To keep v2.1 under the 24,576-byte EIP-170 limit, the convenience views `outputToNative()`, `outputToLp0()` and `outputToLp1()` (address-array reformatting of the routes) were removed. The full routes remain readable through the public getters `outputToNativeRoute(i)`, `outputToLp0Route(i)` and `outputToLp1Route(i)`. No frontend, keeper or dashboard code used the removed views.

4. Behaviour changes

Area	v1	v2 / v2.1
<code>harvest()</code>	Call fee to <code>msg.sender</code>	Unchanged (backwards compatible). Now <code>nonReentrant</code> ; no longer reverts on a killed gauge.
<code>harvest(address)</code>	Existed	Unchanged; recommended for <code>Multicall3</code> / keepers.
Harvest on deposit	Off (all vaults stored <code>false</code>)	On by default for every existing and new vault. The legacy storage slot is reinterpreted as <code>harvestOnDepositDisabled</code> , so no per-vault transaction is needed. <code>harvestOnDeposit()</code> and <code>setHarvestOnDeposit(bool)</code> keep their ABI.

Harvest before withdraw	No	Yes, controlled by the same flag; skipped while paused; cannot revert the withdrawal.
Harvest with no new rewards	No-op	Still deploys idle pair tokens (subject to TWAP guard / zap cap / cooldown).
First deposit	<code>shares = amount</code>	1000 shares locked at <code>0x...dEaD</code> .
UUPS functions	Present but unusable	Removed (<code>upgradeToAndCall</code> , <code>proxiableUUID</code>).
Admin revert strings	Long messages in <code>_setupRoutes</code> , <code>setSwapDeadline</code>	Shortened (bytecode limit). User-facing messages unchanged.
Gas	—	Deposits/withdrawals include a harvest (~0.5–1M gas on Optimism, cents).
Killed gauge (v2.1)	Harvest reverted; LP earned nothing	Harvest unstakes, claims pool swap fees and compounds them; LP stays unstaked; re-staked automatically on revival. New view <code>gaugeAlive()</code> .
Deposit into killed-gauge vault (v2.1)	Reverted	Accepted, held unstaked, earns swap fees.
Route views (v2.1)	<code>outputToNative/Lp0/Lp1()</code>	Removed (I-06); use <code>outputTo*Route(i)</code> .

5. Upgrade safety

- **Storage:** no variable added, removed or reordered. `harvestOnDeposit` (bool) keeps its slot under a new name. OZ 5.x namespaced storage (ERC-7201) for ERC20/Ownable/Pausable/Initializable; the stateless `ReentrancyGuard` uses the same slot `0x9b779b17...5f00` as v1's `ReentrancyGuardUpgradeable`.
- **Implementation lock:** constructor calls `_disableInitializers()`. On-chain: `owner() = 0x0`, `initialize` reverts `InvalidInitialization`. The deployer (EC2 hot wallet `0xE6C3...A340`) has no privileges over the implementation or the vaults.
- **Bytecode:** on-chain runtime code equals the locally built artifact.
- **Rollback:** `factory.upgradeBeacon(0x65c1D7Ae1F840f288c2CCD9a5871fca6497a65f2)` (tested). After rollback `harvestOnDeposit` reads false again.
- **v2.1:** adds no storage variables (only a view, two interface methods and harvest-path logic), so the layout is identical to v2 and v1. On-chain runtime code equals the build; `owner() = 0x0`; the DAO's `upgradeBeacon(v2.1)` call simulates successfully. Rollback to v2 or v1 tested.

6. Test evidence (Optimism mainnet fork, block ~157.6M)

FOUNDRY_PROFILE=velodrome forge test --skip script --match-path test/InfiniteVaultStrategy.fork.t.sol — 12/12 pass against the v2.1 build. The upgrade is executed exactly as on mainnet: DAO Safe → `factory.upgradeBeacon(newImpl)`.

Test	Result	What it proves
<code>test_StorageLayoutPreservedOnAllVaults</code>	PASS	Upgrades the live beacon on a fork (as the DAO Safe); every getter and route of all 6 vaults is byte-identical, and <code>harvestOnDeposit</code> reads true.
<code>test_HarvestDeploysIdleTokens</code>	PASS	ITP/wstETH: idle ITP deployed into LP, TVL increases.
<code>test_ZapIsCappedForIlliquidPools</code>	PASS	ITP price moves $\leq 1.5\%$ per harvest on every vault.
<code>test_HarvestNoArgStillPaysCaller</code>	PASS	Legacy <code>harvest()</code> still pays the call fee to <code>msg.sender</code> .
<code>test_HarvestAllViaMulticallPaysRecipient</code>	PASS	One <code>Multicall3</code> tx harvests all 6 vaults (<code>allowFailure=false</code>); fee reaches recipient, nothing leaks to <code>Multicall3</code> .
<code>test_DepositWithdrawRoundTripAllLiveVaults</code>	PASS	Deposit → 1 day → withdraw on every live vault; user loses no more than the withdrawal fee.
<code>test_WithdrawNeverBlockedByHarvest</code>	PASS	Withdraw with harvest-before-withdraw succeeds on all 6 vaults, including the killed-gauge ITP/DHT.
<code>test_DeadGaugeUnstakesAndCompoundsSwapFees</code>	PASS	v2.1 · ITP/DHT: harvest unstakes all LP from the killed gauge without TVL loss; after real swaps through the pool the next harvest compounds the swap fees and LP grows.
<code>test_DeadGaugeDepositStaysUnstaked</code>	PASS	v2.1 · Deposits into the killed-gauge vault are accepted, stay unstaked and withdraw in full minus the fee.
<code>test_RevivedGaugeRestakes</code>	PASS	v2.1 · When the gauge is alive again, the next harvest re-stakes all LP.
<code>test_FirstDepositMintsDeadShares</code>	PASS	Fresh <code>BeaconProxy</code> : first deposit locks 1000 dead shares; <code>harvestOnDeposit</code> defaults to true.
<code>test_RollbackToPreviousImplementations</code>	PASS	DAO Safe can roll back to v2 (<code>0xE62...79dd</code>) and to v1 (<code>0x65c1...65f2</code>).

7. Upgrade procedure (DAO Safe)

- ## Appendix A – Code diff v1 → v2

```

-- v1 (0x65c1..65f2) InfiniteVaultStrategy.sol
+++ v2 (0xaE62..79dd) InfiniteVaultStrategy.sol

@@ -1,22 +1,27 @@

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.19;

-import {ERC20Upgradeable} from "lib/openzeppelin-contracts-upgradeable/contracts/token/ERC20/ERC20Upgradeable.sol";
-import {IERC20} from "lib/openzeppelin-contracts/contracts/token/ERC20/IERC20.sol";
-import {SafeERC20} from "lib/openzeppelin-contracts/contracts/token/ERC20/utils/SafeERC20.sol";
-import {OwnableUpgradeable} from "lib/openzeppelin-contracts-upgradeable/contracts/access/OwnableUpgradeable.sol";
-import {ReentrancyGuardUpgradeable} from "lib/openzeppelin-contracts-upgradeable/contracts/utils/ReentrancyGuardUpgradeable.sol";
-import {PausableUpgradeable} from "lib/openzeppelin-contracts-upgradeable/contracts/utils/PausableUpgradeable.sol";
-import {UUPSUpgradeable} from "lib/openzeppelin-contracts-upgradeable/contracts/proxy/utils/UUPSUpgradeable.sol";
+import {ERC20Upgradeable} from "@openzeppelin/contracts-upgradeable/token/ERC20/ERC20Upgradeable.sol";
+import {IERC20} from "@openzeppelin/contracts/token/ERC20/IERC20.sol";
+import {SafeERC20} from "@openzeppelin/contracts/token/ERC20/utils/SafeERC20.sol";
+import {Math} from "@openzeppelin/contracts/utils/math/Math.sol";
+import {OwnableUpgradeable} from "@openzeppelin/contracts-upgradeable/access/OwnableUpgradeable.sol";
+// 0Z >=5.5 guard is stateless and uses the same ERC-7201 slot as the deployed ReentrancyGuardUpgradeable.
+import {ReentrancyGuard} from "@openzeppelin/contracts/utils/ReentrancyGuard.sol";
+import {PausableUpgradeable} from "@openzeppelin/contracts-upgradeable/utils/PausableUpgradeable.sol";

import "./interfaces/ISolidlyRouter.sol";
import "./interfaces/ISolidlyPair.sol";
import "./interfaces/IVelodromeGauge.sol";
-import "./interfaces/IERC20Extended.sol";
import "./interfaces/IITP.sol";
-import "./common/IFeeConfig.sol";

-contract InfiniteVaultStrategy is ERC20Upgradeable, OwnableUpgradeable, ReentrancyGuardUpgradeable, PausableUpgradeable, UUPSUpgradeable {
+interface IPoolFactory {
+    function getFee(address pool, bool stable) external view returns (uint256);
+}
+
+/// @dev Beacon implementation: storage layout must stay identical to the deployed one; append only.
+/// UUPSUpgradeable was dropped: it holds no storage and upgradeToAndCall always reverted behind a BeaconProxy.
+contract InfiniteVaultStrategy is ERC20Upgradeable, OwnableUpgradeable, ReentrancyGuard, PausableUpgradeable {
    using SafeERC20 for IERC20;

    struct FeeCategory {

@@ -39,7 +44,8 @@
    address public factory;

    bool public stable;
-    bool public harvestOnDeposit;
+    // Inverted meaning of the legacy `harvestOnDeposit` slot so every existing vault (all stored false) is enabled.
+    bool private harvestOnDepositDisabled;

```

```

uint256 public lastHarvest;

uint256 public withdrawalFee;

uint256 public slippageTolerance;

@@ -50,6 +56,13 @@
    uint256 public constant MAX_FEE = 500 * 1e15;

    uint256 public constant MAX_WITHDRAWAL_FEE = 500;

    uint256 public constant SLIPPAGE_DIVISOR = 10000;

+   uint256 public constant MAX_TWAP_DEVIATION = 300;
+   uint256 public constant TWAP_GRANULARITY = 2;
+   /// @dev Max share of the pair's reserve sold by one zap; ITP pools are thin.
+   uint256 public constant MAX_ZAP_BPS = 50;
+   uint256 public constant ZAP_COOLDOWN = 1 hours;
+   uint256 public constant MIN_SHARES = 1000;
+   address public constant DEAD = 0x0000000000000000000000000000000000000000000000000000000000000000;

    FeeCategory public feeCategory;

    address public infiniteFeeRecipient;

@@ -66,6 +79,11 @@
    event FeeCategoryChanged(uint256 total, uint256 infinite, uint256 call, uint256 burn);

    event WithdrawalFeeChanged(uint256 oldFee, uint256 newFee);

    event SlippageToleranceChanged(uint256 oldTolerance, uint256 newTolerance);

+   event SwapFailed(address indexed tokenIn, uint256 amountIn);
+   event AddLiquidityFailed(uint256 amount0, uint256 amount1);
+   event LiquidityDeferred();
+   event GetRewardFailed();
+   event EarnFailed(uint256 amount);

    /// @custom:oz-upgrades-unsafe-allow constructor
    constructor() {

@@ -85,10 +103,9 @@
    ) public initializer {
        __ERC20_init(_name, _symbol);
        __Ownable_init(msg.sender);
-       __ReentrancyGuard_init();
        __Pausable_init();
-       __UUPSUpgradeable_init();

+
        require(_want != address(0), "Invalid want address");
        require(_gauge != address(0), "Invalid gauge address");
        require(_unirouter != address(0), "Invalid router address");

@@ -117,7 +134,6 @@
        withdrawalFee = 10;

        slippageTolerance = 50;

        swapDeadline = 300;

-       harvestOnDeposit = false;

        _giveAllowances();
    }

@@ -142,9 +158,9 @@
        outputToLpRoute.push(_outputToLpRoute[i]);
    }

    lpToken0 = outputToLpRoute[outputToLpRoute.length - 1].to;

-   require(lpToken0 == token0 || lpToken0 == token1, "LP0 route must end with pair token");
+   require(lpToken0 == token0 || lpToken0 == token1, "Bad LP0 route");
} else {

-   require(output == token0 || output == token1, "Output must be LP token if no LP0 route");

```

```

+         require(output == token0 || output == token1, "Bad LP0 route");
        lpToken0 = output;
    }

@@ -153,13 +169,13 @@
        outputToLp1Route.push(_outputToLp1Route[i]);
    }
    lpToken1 = outputToLp1Route[outputToLp1Route.length - 1].to;
-     require(lpToken1 == token0 || lpToken1 == token1, "LP1 route must end with pair token");
+     require(lpToken1 == token0 || lpToken1 == token1, "Bad LP1 route");
    } else {
-     require(output == token0 || output == token1, "Output must be LP token if no LP1 route");
+     require(output == token0 || output == token1, "Bad LP1 route");
        lpToken1 = output;
    }

-     require(lpToken0 != lpToken1, "LP tokens must be different");
+     require(lpToken0 != lpToken1, "Bad LP tokens");

    if (lpToken0 != token0) {
        address temp = lpToken0;
@@ -167,7 +183,7 @@
        lpToken1 = temp;
    }

-     require(lpToken0 == token0 && lpToken1 == token1, "LP tokens must match pair tokens in correct order");
+     require(lpToken0 == token0 && lpToken1 == token1, "Bad LP tokens");
    }

    function balance() public view returns (uint256) {
@@ -188,7 +204,12 @@

    function deposit(uint256 _amount) public nonReentrant whenNotPaused {
        require(_amount > 0, "Cannot deposit 0");
-
+
+        // Harvest before pricing shares so the depositor can't capture pending rewards.
+        if (!harvestOnDepositDisabled) {
+            _harvest(msg.sender);
+        }
+
        uint256 _pool = balance();
        uint256 _before = IERC20(want).balanceOf(address(this));

@@ -199,16 +220,16 @@

        require(_amount > 0, "No tokens received");

-     if (harvestOnDeposit) {
-         _harvest(msg.sender);
-     }
-
-     uint256 shares = 0;
-     if (totalSupply() == 0) {
-         shares = _amount;
+         require(_amount > MIN_SHARES, "Deposit too small");
+         // Permanently locked shares block first-depositor share inflation.

```

```

+         _mint(DEAD, MIN_SHARES);
+         shares = _amount - MIN_SHARES;
+     } else {
+         shares = (_amount * totalSupply()) / _pool;
+     }
+     require(shares > 0, "Zero shares");

    _earn();
    _mint(msg.sender, shares);

@@ -223,7 +244,11 @@
    function withdraw(uint256 _shares) public nonReentrant {
        require(_shares > 0, "Cannot withdraw 0");
        require(_shares <= balanceOf(msg.sender), "Insufficient shares");
-
+
+     if (!harvestOnDepositDisabled && !paused()) {
+         _harvest(msg.sender);
+     }
+
    uint256 r = (balance() * _shares) / totalSupply();
    _burn(msg.sender, _shares);

@@ -254,6 +279,16 @@
    }
}

+ /// @dev Harvest-path variant: a killed gauge must not block harvest-before-withdraw.
+ function _tryEarn() internal {
+     uint256 wantBal = IERC20(want).balanceOf(address(this));
+     if (wantBal > 0) {
+         try IVelodromeGauge(gauge).deposit(wantBal, address(this)) {} catch {
+             emit EarnFailed(wantBal);
+         }
+     }
+ }
+
    function _withdrawFromGauge(uint256 _amount) internal {
        uint256 wantBal = IERC20(want).balanceOf(address(this));
        if (wantBal < _amount) {
@@ -261,25 +296,31 @@
    }
}

- function harvest() external whenNotPaused {
+ function harvest() external nonReentrant whenNotPaused {
+     _harvest(msg.sender);
+ }

- function harvest(address callFeeRecipient) external whenNotPaused {
+ /// @notice Use this from Multicall3/batchers so the call fee goes to `callFeeRecipient`, not the batcher.
+ function harvest(address callFeeRecipient) external nonReentrant whenNotPaused {
+     require(callFeeRecipient != address(0), "Invalid fee recipient");
+     _harvest(callFeeRecipient);
+ }

+ /// @dev Never reverts on external failures, because deposit() and withdraw() call it.
    function _harvest(address callFeeRecipient) internal {

```

```

- IVelodromeGauge(gauge).getReward(address(this));
- uint256 outputBal = IERC20(output).balanceOf(address(this));
+ try IVelodromeGauge(gauge).getReward(address(this)) {} catch {
+     emit GetRewardFailed();
+ }

- if (outputBal > 0) {
+ if (IERC20(output).balanceOf(address(this)) > 0) {
+     _chargeFees(callFeeRecipient);
- _addLiquidity();
- uint256 wantHarvested = IERC20(want).balanceOf(address(this));
- _earn();
+ }
+ // Runs even without new rewards so idle pair tokens from earlier harvests get deployed.
+ _addLiquidity();

+ uint256 wantHarvested = IERC20(want).balanceOf(address(this));
+ if (wantHarvested > 0) {
+     _tryEarn();
+     lastHarvest = block.timestamp;
+     emit StratHarvest(msg.sender, wantHarvested, balance());
+ }

@@ -331,91 +372,178 @@
+     emit ChargedFees(callFeeAmount, infiniteFeeAmount, burnFeeAmount);
+ }
+ } catch {
+     emit SwapFailed(output, toNative);
+     return;
+ }
+ }

function _addLiquidity() internal {
+ if (!_isPoolPriceSane()) {
+     emit LiquidityDeferred();
+     return;
+ }
+
+ uint256 outputBal = IERC20(output).balanceOf(address(this));
- if (outputBal == 0) return;
-
- uint256 lp0Amt = outputBal / 2;
- uint256 lp1Amt = outputBal - lp0Amt;
+ if (outputBal > 0) {
+     uint256 lp0Amt = _rewardSplit0(outputBal);
+     if (lpToken0 != output && lp0Amt > 0) _swap(lp0Amt, outputToLp0Route);
+     if (lpToken1 != output && outputBal > lp0Amt) _swap(outputBal - lp0Amt, outputToLp1Route);
+ }

- if (stable) {
-     try IERC20Extended(lpToken0).decimals() returns (uint decimals0) {
-         try IERC20Extended(lpToken1).decimals() returns (uint decimals1) {
-             uint256 lp0Decimals = 10**decimals0;
-             uint256 lp1Decimals = 10**decimals1;
+ _addBalancedLiquidity();
+ if (block.timestamp >= lastHarvest + ZAP_COOLDOWN && _zapExcess()) {
+     _addBalancedLiquidity();

```

```

+     }
+ }

-         uint256 out0 = lpToken0 != output && outputToLp0Route.length > 0 ?
-             _getAmountOut(lp0Amt, outputToLp0Route) * 1e18 / lp0Decimals :
-             lp0Amt;
-         uint256 out1 = lpToken1 != output && outputToLp1Route.length > 0 ?
-             _getAmountOut(lp1Amt, outputToLp1Route) * 1e18 / lp1Decimals :
-             lp1Amt;
+     /// @dev Reward amount to route to lpToken0 so that, together with idle balances, both sides land on the pool ratio.
+     function _rewardSplit0(uint256 outputBal) internal view returns (uint256) {
+         uint256 q0 = lpToken0 == output ? outputBal : _getAmountOut(outputBal, outputToLp0Route);
+         uint256 q1 = lpToken1 == output ? outputBal : _getAmountOut(outputBal, outputToLp1Route);
+         // A dead route: send everything to the side that works; _zapExcess converts it via the pair.
+         if (q0 == 0 && q1 == 0) return outputBal / 2;
+         if (q1 == 0) return outputBal;
+         if (q0 == 0) return 0;

-         if (out0 > 0 && out1 > 0) {
-             try ISolidlyRouter(unirouter).quoteAddLiquidity(
-                 lpToken0, lpToken1, stable, factory, out0, out1
-             ) returns (uint256 amountA, uint256 amountB, uint256) {
-                 if (amountA > 0 && amountB > 0) {
-                     amountA = amountA * 1e18 / lp0Decimals;
-                     amountB = amountB * 1e18 / lp1Decimals;
-                     if (out1 > 0 && amountA > 0) {
-                         uint256 ratio = out0 * 1e18 / out1 * amountB / amountA;
-                         lp0Amt = outputBal * 1e18 / (ratio + 1e18);
-                         lp1Amt = outputBal - lp0Amt;
-                     }
-                 }
-             } catch {
-             }
-         } catch {
-         }
-     } catch {
-     }
- }

+         uint256 idle0 = IERC20(lpToken0).balanceOf(address(this)) - (lpToken0 == output ? outputBal : 0);
+         uint256 idle1 = IERC20(lpToken1).balanceOf(address(this)) - (lpToken1 == output ? outputBal : 0);
+         (uint256 reserve0, uint256 reserve1) = ISolidlyPair(want).getReserves();
+
+         // f = (R0*(idle1+q1) - R1*idle0) / (R1*q0 + R0*q1), clamped to [0, 1]
+         uint256 plus = reserve0 * (idle1 + q1);
+         uint256 minus = reserve1 * idle0;
+         if (plus <= minus) return 0;
+         uint256 den = reserve1 * q0 + reserve0 * q1;
+         if (plus - minus >= den) return outputBal;
+         return Math.mulDiv(outputBal, plus - minus, den);
+     }
+
+     /// @dev Deposits the largest pool-ratio-matched amounts; min amounts derive from the quote, not raw balances.
+     function _addBalancedLiquidity() internal {
+         uint256 bal0 = IERC20(lpToken0).balanceOf(address(this));
+         uint256 bal1 = IERC20(lpToken1).balanceOf(address(this));
+         if (bal0 == 0 || bal1 == 0) return;
+
+         (uint256 amt0, uint256 amt1) = _quoteAddLiquidity(bal0, bal1);

```

```

+     if (amt0 == 0 || amt1 == 0) return;
+
+     try ISolidlyRouter(unirouter).addLiquidity(
+         lpToken0,
+         lpToken1,
+         stable,
+         amt0,
+         amt1,
+         _applySlippage(amt0),
+         _applySlippage(amt1),
+         address(this),
+         block.timestamp + swapDeadline
+     ) {} catch {
+         emit AddLiquidityFailed(amt0, amt1);
+     }
+ }

-     if (lpToken0 != output && lp0Amt > 0 && outputToLp0Route.length > 0) {
-         uint256 expectedOut = _getAmountOut(lp0Amt, outputToLp0Route);
-         if (expectedOut > 0) {
-             uint256 minOut = expectedOut * (SLIPPAGE_DIVISOR - slippageTolerance) / SLIPPAGE_DIVISOR;
-             try ISolidlyRouter(unirouter).swapExactTokensForTokens(
-                 lp0Amt, minOut, outputToLp0Route, address(this), block.timestamp + swapDeadline
-             ) {} catch {
-                 return;
-             }
-         }
-     }

+ /// @dev Swaps part of the one-sided leftover through the pair itself so it can be deposited.
+ function _zapExcess() internal returns (bool) {
+     uint256 bal0 = IERC20(lpToken0).balanceOf(address(this));
+     uint256 bal1 = IERC20(lpToken1).balanceOf(address(this));
+     if (bal0 == 0 && bal1 == 0) return false;
+
+     uint256 used0;
+     uint256 used1;
+     if (bal0 > 0 && bal1 > 0) {
+         (used0, used1) = _quoteAddLiquidity(bal0, bal1);
+         if (used0 == 0 && used1 == 0) return false;
+     }

-     if (lpToken1 != output && lp1Amt > 0 && outputToLp1Route.length > 0) {
-         uint256 expectedOut = _getAmountOut(lp1Amt, outputToLp1Route);
-         if (expectedOut > 0) {
-             uint256 minOut = expectedOut * (SLIPPAGE_DIVISOR - slippageTolerance) / SLIPPAGE_DIVISOR;
-             try ISolidlyRouter(unirouter).swapExactTokensForTokens(
-                 lp1Amt, minOut, outputToLp1Route, address(this), block.timestamp + swapDeadline
-             ) {} catch {
-                 return;
-             }
-         }
-     }

+     bool zeroForOne = bal0 > used0;
+     uint256 excess = zeroForOne ? bal0 - used0 : bal1 - used1;
+     if (excess == 0) return false;
+
+     (uint256 reserve0, uint256 reserve1) = ISolidlyPair(want).getReserves();
+     uint256 reserveIn = zeroForOne ? reserve0 : reserve1;
+     uint256 amountIn = Math.min(_zapSwapAmount(excess, reserveIn), reserveIn * MAX_ZAP_BPS / SLIPPAGE_DIVISOR);

```

```

+     if (amountIn == 0) return false;
+
+     ISolidlyRouter.Route[] memory route = new ISolidlyRouter.Route[](1);
+     route[0] = ISolidlyRouter.Route({
+         from: zeroForOne ? lpToken0 : lpToken1,
+         to: zeroForOne ? lpToken1 : lpToken0,
+         stable: stable,
+         factory: factory
+     });
+     return _swap(amountIn, route);
+ }
+
+ /// @dev Optimal single-sided zap for x*y=k: s = (sqrt(r(r(2-f)^2 + 4(1-f)E)) - r(2-f)) / (2(1-f)).
+ function _zapSwapAmount(uint256 excess, uint256 reserveIn) internal view returns (uint256) {
+     if (stable || reserveIn == 0) return excess / 2;
+
+     uint256 fee = _poolFee();
+     uint256 a = 20_000 - fee;
+     uint256 b = 10_000 - fee;
+     return (Math.sqrt(reserveIn * (reserveIn * a * a + 40_000 * b * excess)) - reserveIn * a) / (2 * b);
+ }
+
+ function _poolFee() internal view returns (uint256) {
+     try IPoolFactory(factory).getFee(want, stable) returns (uint256 f) {
+         if (f < 1000) return f;
+     } catch {}
+     return 30;
+ }
+
+ function _quoteAddLiquidity(uint256 bal0, uint256 bal1) internal view returns (uint256, uint256) {
+     try ISolidlyRouter(unirouter).quoteAddLiquidity(lpToken0, lpToken1, stable, factory, bal0, bal1) returns (
+         uint256 amt0, uint256 amt1, uint256
+     ) {
+         return (amt0, amt1);
+     } catch {
+         return (0, 0);
+     }
+ }
+
-     uint256 lp0Bal = IERC20(lpToken0).balanceOf(address(this));
-     uint256 lp1Bal = IERC20(lpToken1).balanceOf(address(this));
-
-     if (lp0Bal > 0 && lp1Bal > 0) {
-         uint256 minLp0 = lp0Bal * (SLIPPAGE_DIVISOR - slippageTolerance) / SLIPPAGE_DIVISOR;
-         uint256 minLp1 = lp1Bal * (SLIPPAGE_DIVISOR - slippageTolerance) / SLIPPAGE_DIVISOR;
-
-         try ISolidlyRouter(unirouter).addLiquidity(
-             lpToken0, lpToken1, stable, lp0Bal, lp1Bal, minLp0, minLp1, address(this), block.timestamp + swapDeadline
-         ) {} catch {
-
-         }
+     function _swap(uint256 amountIn, ISolidlyRouter.Route[] memory route) internal returns (bool) {
+         uint256 expectedOut = _getAmountOut(amountIn, route);
+         if (expectedOut == 0) return false;
+
+         try ISolidlyRouter(unirouter).swapExactTokensForTokens(
+             amountIn, _applySlippage(expectedOut), route, address(this), block.timestamp + swapDeadline
+         ) {
+             return true;

```

```

+     } catch {
+         emit SwapFailed(route[0].from, amountIn);
+         return false;
+     }
+ }

+ function _applySlippage(uint256 amount) internal view returns (uint256) {
+     return amount * (SLIPPAGE_DIVISOR - slippageTolerance) / SLIPPAGE_DIVISOR;
+ }
+
+ /// @dev harvest() is permissionless, so refuse to trade/deposit while the pair's spot price is off its TWAP.
+ function _isPoolPriceSane() internal view returns (bool) {
+     (uint256 reserve0,,) = ISolidlyPair(want).getReserves();
+     uint256 amountIn = reserve0 / 1000;
+     if (amountIn == 0) return false;
+
+     uint256 spot;
+     try ISolidlyPair(want).getAmountOut(amountIn, lpToken0) returns (uint256 out) {
+         spot = out;
+     } catch {
+         return false;
+     }
+
+     try ISolidlyPair(want).quote(lpToken0, amountIn, TWAP_GRANULARITY) returns (uint256 twap) {
+         // quote() excludes the swap fee that getAmountOut() charges.
+         twap = twap * (SLIPPAGE_DIVISOR - _poolFee()) / SLIPPAGE_DIVISOR;
+         if (twap == 0) return false;
+         uint256 diff = spot > twap ? spot - twap : twap - spot;
+         return diff * SLIPPAGE_DIVISOR <= twap * MAX_TWAP_DEVIATION;
+     } catch {
+         return false;
+     }
+ }
+
+ function _getAmountOut(uint256 amountIn, ISolidlyRouter.Route[] memory route) internal view returns (uint256) {
+     if (route.length == 0 || amountIn == 0) return 0;

```

@@ -453,8 +581,13 @@

```

    return totalFeesInNative * feeCategory.call / totalFeeShares;
}

```

```

+ function harvestOnDeposit() external view returns (bool) {
+     return !harvestOnDepositDisabled;
+ }
+
+ /// @notice Also controls the harvest that runs before withdraw().
+ function setHarvestOnDeposit(bool _harvestOnDeposit) external onlyOwner {
-     harvestOnDeposit = _harvestOnDeposit;
+     harvestOnDepositDisabled = !_harvestOnDeposit;
+ }

```

```

    function setWithdrawalFee(uint256 _withdrawalFee) external onlyOwner {

```

@@ -472,7 +605,7 @@

```

    }

```

```

    function setSwapDeadline(uint256 _swapDeadline) external onlyOwner {
-     require(_swapDeadline >= 60 && _swapDeadline <= 3600, "Deadline must be between 1 minute and 1 hour");

```

```

+     require(_swapDeadline >= 60 && _swapDeadline <= 3600, "Bad deadline");
        swapDeadline = _swapDeadline;
    }

@@ -511,7 +644,7 @@
    function unpause() external onlyOwner {
        _unpause();
        _giveAllowances();
-     _earn();
+     _tryEarn();
    }

    function inCaseTokensGetStuck(address _token) external onlyOwner {

@@ -557,6 +690,4 @@
    function outputToLp1() external view returns (address[] memory) {
        return _solidlyToRoute(outputToLp1Route);
    }

-
-     function _authorizeUpgrade(address newImplementation) internal override onlyOwner {}
    }

\ No newline at end of file
--- v1 ISolidlyPair.sol
+++ v2 ISolidlyPair.sol

@@ -10,4 +10,5 @@
    function getReserves() external view returns (uint112 reserve0, uint112 reserve1, uint32 blockTimestampLast);
    function stable() external view returns (bool);
    function getAmountOut(uint256 amountIn, address tokenIn) external view returns (uint256);
+     function quote(address tokenIn, uint256 amountIn, uint256 granularity) external view returns (uint256 amountOut);
    }

\ No newline at end of file

```

Appendix B — Code diff v2 → v2.1

The v2 side was reconstructed from source and compiles to exactly the on-chain v2 runtime bytecode.

```

--- v2 (0xaE62...79dd)
+++ v2.1 (0x3647...2944)

@@ -19,6 +19,10 @@
    function getFee(address pool, bool stable) external view returns (uint256);
    }

+interface IVoter {
+     function isAlive(address gauge) external view returns (bool);
+}
+
+    /// @dev Beacon implementation: storage layout must stay identical to the deployed one; append only.
+    /// UUPSUpgradeable was dropped: it holds no storage and upgradeToAndCall always reverted behind a BeaconProxy.
+    contract InfiniteVaultStrategy is ERC20Upgradeable, OwnableUpgradeable, ReentrancyGuard, PausableUpgradeable {

@@ -274,7 +278,7 @@
    function _earn() internal {
        uint256 wantBal = IERC20(want).balanceOf(address(this));
-     if (wantBal > 0) {
+     if (wantBal > 0 && gaugeAlive()) {
        IVelodromeGauge(gauge).deposit(wantBal, address(this));
    }
}

```

@@ -308,22 +312,43 @@

```
    /// @dev Never reverts on external failures, because deposit() and withdraw() call it.
    function _harvest(address callFeeRecipient) internal {
-       try IVelodromeGauge(gauge).getReward(address(this)) {} catch {
-           emit GetRewardFailed();
+       bool alive = gaugeAlive();
+       if (alive) {
+           try IVelodromeGauge(gauge).getReward(address(this)) {} catch {
+               emit GetRewardFailed();
+           }
+       } else {
+           // Killed gauge pays no emissions and keeps the swap fees; hold LP unstaked and earn the fees instead.
+           uint256 staked = balanceOfPool();
+           if (staked > 0) {
+               try IVelodromeGauge(gauge).withdraw(staked) {} catch {}
+           }
+           try ISolidlyPair(want).claimFees() {} catch {}
        }

        if (IERC20(output).balanceOf(address(this)) > 0) {
            _chargeFees(callFeeRecipient);
        }
+       uint256 wantBefore = IERC20(want).balanceOf(address(this));
        // Runs even without new rewards so idle pair tokens from earlier harvests get deployed.
        _addLiquidity();
```

```
-       uint256 wantHarvested = IERC20(want).balanceOf(address(this));
+       uint256 wantHarvested = IERC20(want).balanceOf(address(this)) - wantBefore;
+       if (alive) _tryEarn();
        if (wantHarvested > 0) {
-           _tryEarn();
            lastHarvest = block.timestamp;
            emit StratHarvest(msg.sender, wantHarvested, balance());
        }
+   }
+
+   /// @notice False once Velodrome kills the gauge; the vault then holds its LP unstaked.
+   function gaugeAlive() public view returns (bool) {
+       try IVelodromeGauge(gauge).voter() returns (address voter) {
+           try IVoter(voter).isAlive(gauge) returns (bool alive) {
+               return alive;
+           } catch {}
+       } catch {}
+       return true;
    }
```

```
    function _chargeFees(address callFeeRecipient) internal {
```

@@ -669,25 +694,4 @@

```
        IERC20(lpToken0).forceApprove(unirouter, 0);
        IERC20(lpToken1).forceApprove(unirouter, 0);
    }

-
-   function _solidlyToRoute(ISolidlyRouter.Route[] memory _route) internal pure returns (address[] memory) {
-       address[] memory route = new address[](_route.length + 1);
-       route[0] = _route[0].from;
-       for (uint i; i < _route.length; ++i) {
-           route[i + 1] = _route[i].to;
```

```

-     }
-
-     return route;
- }
-
-
- function outputToNative() external view returns (address[] memory) {
-     return _solidlyToRoute(outputToNativeRoute);
- }
-
-
- function outputToLp0() external view returns (address[] memory) {
-     return _solidlyToRoute(outputToLp0Route);
- }
-
-
- function outputToLp1() external view returns (address[] memory) {
-     return _solidlyToRoute(outputToLp1Route);
- }
-
- }
-
\ No newline at end of file
--- v2 IVelodromeGauge.sol
+++ v2.1 IVelodromeGauge.sol
@@ -10,4 +10,5 @@
     function balanceOf(address user) external view returns (uint256);
     function rewardToken() external view returns (address);
     function rewardPerToken() external view returns (uint256);
+ function voter() external view returns (address);
}
\ No newline at end of file
--- v2 ISolidlyPair.sol
+++ v2.1 ISolidlyPair.sol
@@ -11,4 +11,5 @@
     function stable() external view returns (bool);
     function getAmountOut(uint256 amountIn, address tokenIn) external view returns (uint256);
     function quote(address tokenIn, uint256 amountIn, uint256 granularity) external view returns (uint256 amountOut);
+ function claimFees() external returns (uint256 claimed0, uint256 claimed1);
}
\ No newline at end of file

```